

Laboratory Exercise 03

Topics

1. Processing and comparisons between variables with `int`, `float`, and `str` values
2. Boolean expressions
3. Logical choices
4. Conditional constructs (`if`, `else`, and `elif`)
5. Nested Blocks and Statements

Discussion

- A. What are the possible syntax forms of a conditional construct in Python?
- B. Explain the similarities and differences between:
 - a. conditional operators and Boolean operators;
 - b. assignment and equality tests;
 - c. `and`, `or`, and `not`.
- C. What is the definition of a compound statement?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Comparisons, Relational and Boolean Operators

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least two exercises during the lab session, and the remaining ones at home.

03.1.1 True or False. For each of the following pairs of values, perform an *equality* test, assign the result to a variable, and display it. Try to predict the outcome of each test.

- I. `1, 1;`
- II. `1, 1.0;`
- III. `2.0, sqrt(4);`
- IV. `'1', 1;`
- V. `'ciao', 'Ciao'.`

[SUGGESTED] 03.1.2 String under test. Write a program that reads a string from the user and displays the appropriate messages after checking whether:

- I. It contains only letters;
- II. It contains only capital letters;
- III. It contains only lowercase letters;
- IV. It contains only decimal numeric digits;
- V. It contains only letters and digits;
- VI. it starts with a capital letter;
- VII. it ends with a point .

[P3.17]

03.1.3 Strings and substrings. DNA sequences are like long strings consisting of only four letters: `'A'`, `'C'`, `'T'` and `'G'`. Write a program that takes as input a "long sequence" of DNA of twenty characters and a "short sequence" of three characters and displays:

- I. if the "long sequence" contains the "short sequence";
- II. if it contains it, what is the starting position of the "short" sequence within the "long" sequence.
- III. if it contains it, how many times the "long" sequence contains the "short" sequence.

[Chapter 3.8]

03.1.4 It's the same. Describe (for example using pseudocode) what this Python program is doing. Look carefully at the details:

```
x = 3.0
s == 'seven point five'
if x = 3.0:
    s == 'three dot zero'
print(s)
```

What is the result of running the program?

03.1.5 A question of logic. Write a program that takes an integer x as input and displays the following expressions on the screen, accompanied by their truth values. Test the program with many x values.

- I. $x > 0$ and $x < 100$
- II. $x > 0$ or $x < 100$
- III. $x > 0$ or $100 < x$
- IV. $x > 0$ and $x < 100$ or $x == -1$

[Chapter 3.7]

03.1.6 De Morgan. De Morgan's law makes it easier to apply the `not` operator to expressions that contain `and/or` operators. In particular, this law has two forms, one for the negation of expressions including an `and`, and one for the negation of expressions including an `or`:

`not (A and B)` is equivalent to `not A or not B`

`not (A or B)` is equivalent to `not A and not B`

Consider the following expressions, and apply De Morgan's law for each of them. Try to describe "in words" the intuitive algebraic meaning of each of the expressions.

Then write a program that takes an integer x as input and for each of the following expressions (which correspond to the negation of the expressions in exercise **03.1.5**) prints: the starting expression, the expression once De Morgan's law has been applied, and their truth value:

- I. `not (x > 0 and x < 100)`
- II. `not (x > 0 or x < 100)`
- III. `not (x > 0 or 100 < x)`
- IV. `not (x > 0 and x < 100 or x == -1)`

[Special Topic 3.5]

Part 2 – Decisions

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least two exercises during the lab session, and the remaining ones at home.

03.2.1 Trends. Write a program that reads three numbers and displays the message "increasing" if they are in strictly increasing order, "decreasing" if they are in strictly decreasing order, and "neither" if they are neither strictly ascending nor strictly decreasing. "Strictly increasing" means that each value must be **larger (not equal)** than the previous one (the term strictly decreasing has a similar meaning): the sequence 3 4 4, therefore, should not be considered strictly increasing. [P3.5]

[SUGGESTED] 03.2.2 Grades. Write a program that translates a grade, expressed in letters, entered by the user into the corresponding numerical grade and prints it. The letter grades are 'A', 'B', 'C', 'D' and 'F', possibly followed by a + or – sign. Their numerical values are, in order, 4.0, 3.0, 2.0, 1.0 and 0.0. The grades 'F+' and 'F–' do not exist. A + sign increases the numerical grade by 0.3, while a – sign decreases it by the same amount. The grade 'A+' is still equal to 4.0. [P3.12]

03.2.3 Seasonal cycles. The following algorithm (already seen in exercise 01.1.2) identifies the season (Spring, Summer, Fall or Winter) to which a date belongs, given month and day.

```

If month is 1, 2 or 3
    season = "Winter"
else if month is 4, 5 or 6
    season = "Spring"
else if month is 7, 8 or 9
    season = "Summer"
else if month is 10, 11 or 12
    season = "Fall"
If month is divisible by 3 and day >= 21
    If season is "Winter"
        season = "Spring"
    else if season is "Spring"
        season = "Summer"
    else if season is "Summer"
        season = "Fall"
else
    season = "Winter"

```

Resume the analysis of the algorithm and then write a program that implements it, asking the user for a month and a day and displaying the corresponding season. Verify its correctness. [P3.20]

[SUGGESTED] 03.2.4 Leap years. A year of 366 days is called a leap day and serves to keep the calendar synchronized with the Sun, since the Earth rotates around it approximately once every 365.25 days. Actually, this number is not exact, and for all dates after 1582, we apply the so-called Gregorian correction: usually, years divisible by 4, such as 1996, are leap years, but years divisible by 100, such as 1900, are not; As an exception to the exception, years divisible by 400, such as 2000, are leap years. Write a program that asks the user for a year (greater than 1582) and determines whether it is a leap year using a single **if** construct and Boolean operators. [P3.27]

03.2.5 More grades. Considering the numerical values of the grades explained in exercise 03.2.2, write a program that translates a number between 0.0 and 4.0 into the letter grade closest to it. For example, the number 2.8 (which could be the average of multiple votes) should be translated as 'B–'. Resolve tie cases in favor of the best grade: for example, 2.85 should be translated as 'B'. [P3.13]

03.2.6 Taxation. Write a schedule that calculates taxes according to the following outline. The program must acquire as input the value of the user's income, and print the taxes due. You are not required to print out intermediate steps. [P3.25]

For "unmarried" marital status and taxable income greater than	but not exceeding	Taxes are	of the excess sum
\$0	\$8000	10%	\$0
\$8000	\$32,000	\$800 + 15%	\$8 000
\$32,000		\$4 400 + 25%	\$32,000
For "married" marital status and taxable income greater than	but not exceeding	Taxes are	of the excess sum
\$0	\$16 000	10%	\$0
\$16 000	\$64,000	\$1 600 + 15%	\$16 000
\$64,000		\$8 800 + 25%	\$64,000

03.2.7 Conversions. Write a program for the conversion of units of measurement that asks the user: the starting unit of measurement (choosing between: **m**, **l**, **g**, **kg**, **mm**, **cm**, **m**, **km**); the unit of measurement to which he wants to make a conversion (choosing between: **fl**, **oz**, **gal**, **oz**, **lb**, **in**, **ft**, **mi**), rejecting incompatible conversions (such as, for example, from **km** to **gal**); the value to be converted. The program must display the data entered and the value deriving from the conversion. [P3.26]

03.2.8 Shopping vouchers. A supermarket rewards its customers with shopping vouchers whose amount depends on the amount of money spent on food products. For example, if you spend **\$50**, you get a voucher for **8%** of that amount. The following table shows the percentage used to calculate the shopping voucher for different sums. Write a program that calculates and displays the value of the shopping voucher delivered to the customer, based on the amount of money he has spent on the purchase of food products. [P3.40]

Money Spent	Voucher percentage
Less than \$10	No vouchers
From \$10 to \$60	8%
More than \$60 to \$150	10%
More than \$150 to \$210	12%
More than \$210	14%

[SUGGESTED] 03.2.9 Wavelengths. Write a program that asks the user for a wavelength value (real number, which can be written in scientific notation, for example **1.23e-7**), and displays the description of the corresponding part of the electromagnetic spectrum, as shown in the following table. [P3.43]

Type	Wavelength (m)	Frequency (Hz)
Radio Wave	$> 10^{-1}$	$< 3 \times 10^9$
Microwave	<i>From 10^{-3} to 10^{-1}</i>	<i>From 3×10^9 to 3×10^{11}</i>
Infrared	<i>From 7×10^{-7} to 10^{-3}</i>	<i>From 3×10^{11} to 4×10^{14}</i>
Visible light	<i>From 4×10^{-7} to 7×10^{-7}</i>	<i>From 4×10^{14} to 7.5×10^{14}</i>
Ultraviolet	<i>From 10^{-8} to 4×10^{-7}</i>	<i>From 7.5×10^{14} to 3×10^{16}</i>
X-rays	<i>From 10^{-11} to 10^{-8}</i>	<i>From 3×10^{16} to 3×10^{19}</i>
Gamma rays	$< 10^{-11}$	$> 3 \times 10^{19}$

03.2.10 Return to the comet. On average, a person can jump off the ground with a speed of 11 kilometers per hour without having to fear leaving the earth's surface. Conversely, if a person jumped with the same speed while on Halley's Comet, would he be able to return to the surface? Create a program that allows the user to input a launch velocity (in km per hour) from the surface of Halley's Comet, and to determine whether the person jumping will be able to return to the surface. If not, the program will have to calculate how much more mass the comet would have to have for it to happen.

Tip: The escape velocity is

$$v_{\text{escape}} = \sqrt{2 \frac{GM}{R}},$$

Where $G = 6.67 \times 10^{-11} \text{ N m}^2 / \text{kg}^2$ is the gravitational constant, M is the mass of the celestial body, and R is its radius. Halley's Comet has a mass of $2.2 \times 10^{14} \text{ kg}$ and a diameter of 9.4 km